

Seguridad en la autenticación

Llave USB–TOTP: con ESP32 y PAM de Linux

Javier Rodriguez (@emb0scad0)
Github: https://github.com/pinguytaz
Web: www.pinguytaz.net

30 Octubre 2026

Biografia

Inicios con un sencillo ZX81 y un Spectrum, vinculado al desarrollo cercano al hardware y tecnologías emergentes, gran interes en los sistemas embebidos.

Publicación de artículos en revistas especializadas: ZX, TodoSpectrum, RPP, JavaMagazine.

Autor del libro “De 0 a 100 con Arduino y ESP32”



- Datos personales:
  - Nombre: Fco. Javier Rodriguez
  - Telegram: @emb0scad0
  - Github: https://github.com/pinguytaz
  - Web: https://www.pinguytaz.net

Objetivos

- Objetivo
- Protocolo encriptado
- PAM (Pluggable Authentication Modules)
  - ¿Que es?
  - Configuración
  - Programación
- ESP32
  - Particiones (Datos Seguros del TOTP)
  - Comunicaciones

Objetivo

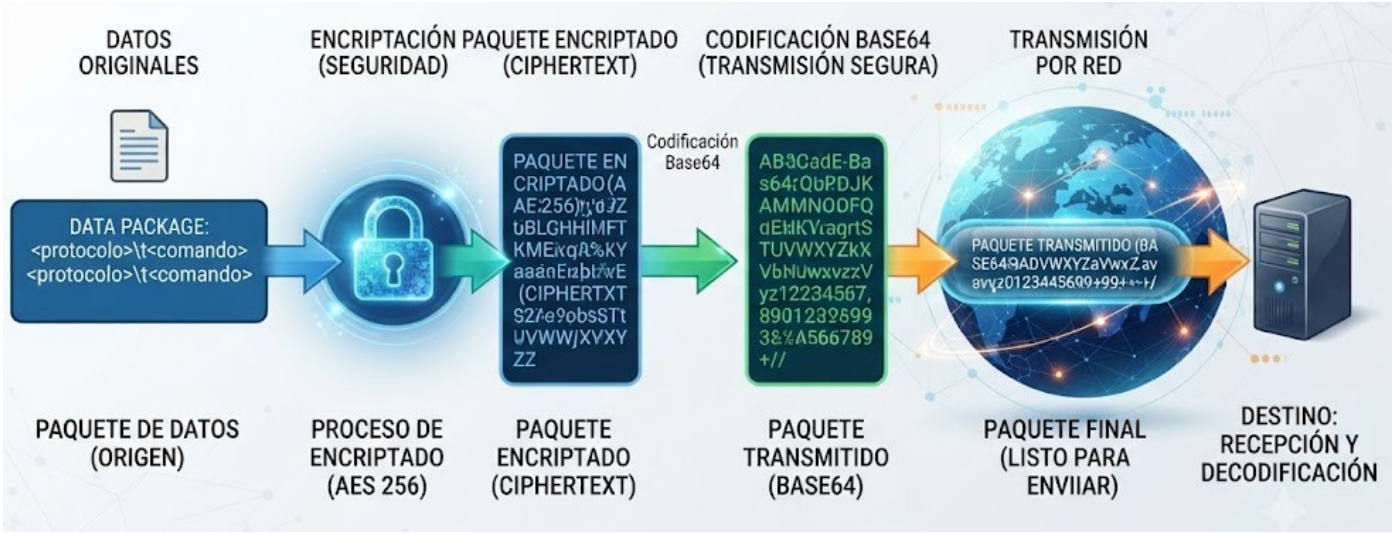
Desarrollo una llave USB-TOTP con un **ESP32** y modulo **PAM** para la comunicación y autentificación.

TOTP(Time-based One-Time Password): Contraseña de un solo uso basada en el tiempo.

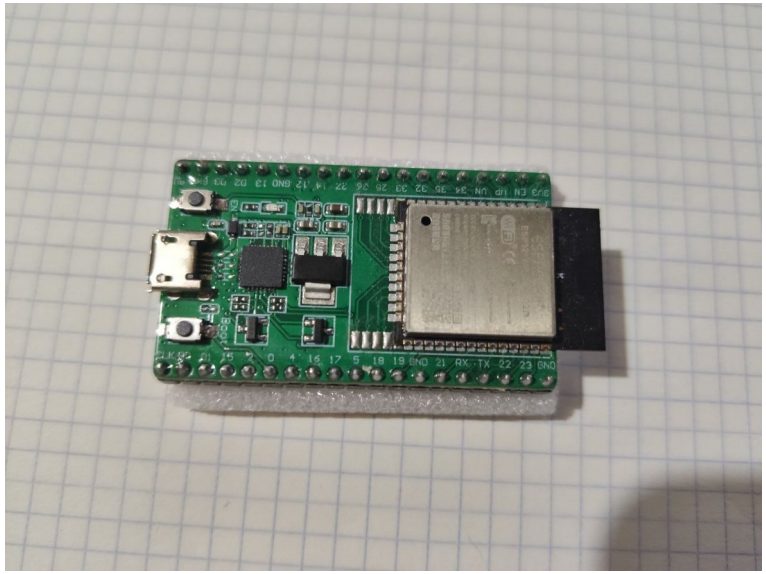
Protocolo encriptado

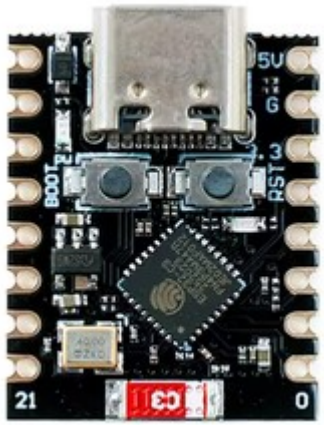
PAM (**Pluggable Authentication Modules**) Módulos para servicios que facilita no tener que implementar el mecanismo de acceso. Simplemente pasar las credenciales a PAM y este se encargue de indicar si el usuario tiene acceso a no. Personalizando el modo de funcionamiento.

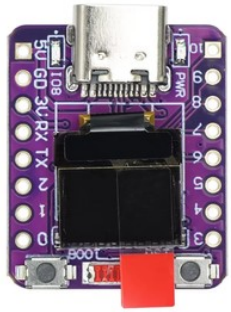
Flujo


DATOS ORIGINALES
ENCRIPTACIÓN (SEGURIDAD)
PAQUETE ENCRIPTADO (CIPHERTEXT)
CODIFICACIÓN BASE64 (TRANSMISIÓN SEGURA)
TRANSMISIÓN POR RED
DATA PACKAGE:
<protocolo>\t<comando>
<protocolo>\t<comando>
Codificación Base64
PAQUETE DE DATOS (ORIGEN)
PROCESO DE ENCRIPTADO (AES 256)
PAQUETE ENCRIPTADO (CIPHERTEXT)
PAQUETE TRANSMITIDO (BASE64)
PAQUETE FINAL (LISTO PARA ENVIIAR)
DESTINO: RECEPCIÓN Y DECODIFICACIÓN

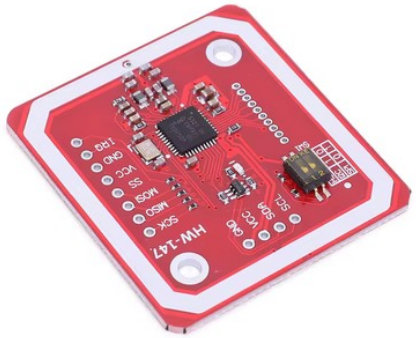
ESP32







- Memoria
- WIFI
- Conexión NFC
- Librerías encriptación



- Dispositivos con pantalla

ESP32

- Programación IDE-Arduino
- Preferencias en partición ‘nvs’ (ID y Usuario)
- #include <mbedtls/aes.h> Encriptación AES
- #include <mbedtls/base64.h> Base64
- Comunicación por USB, DEBUG en otro puerto
- Futuros proyectos
  - WIFI
  - Conexión NFC

ESP32 memoria Flash

Tipos de particiones

- NVS (Clave-Valor) se puede encriptar
- spiffs, littlefs o fat (Sistemas de Archivos)

  Encriptaríamos los ficheros

```
# Name,   Type, SubType,  Offset,   Size,
nvs,      data, nvs,      0x9000,   0x5000,    # (20 KB para NVS)
Otadata,  data, ota,      0xe000,   0x2000,    # (8 KB para estado OTA)
App0,     app,  ota_0,    0x10000,  0x140000,  # (1.25 MB para tu código)
App1,     app,  ota_1,    0x150000, 0x140000,  # (1.25 MB para actualizaciones OTA)
Spiffs,   data, spiffs,   0x290000, 0x170000,  # (1.4 MB para archivos web/datos)
```

PAM

Desacopla la lógica de autenticación de las aplicaciones (login, sshd, sudo, gdm, ligthdm, etc.), permitiendo que el administrador del sistema configure cómo y cuándo se valida.

Ventajas:

- **Flexibilidad**: Permite dar diferentes métodos de autentificación sin modificar el código de nuestras aplicaciones.
- **Seguridad**: Las actualizaciones de los métodos de autentificación no altera las aplicaciones.
- **Personalización**: Realizar nuestros módulos

PAM (configuración)

Módulo de autentificación, que solicita a la llave la clave TOTP y el usuario de acceso.

/etc/pam.d/login

```
#%PAM-1.0
# Comentar para eliminar la solicitud de Llave
auth    requisite       TOTP_Simple.so <puerto< <tty rescate> <usuario rescate>
```

Auth: Autentificación del usuario.

Requisite: Debe pasar, si falla salta proceso.

[required]: Si falla continua aunque al final fallo.

PAM (configuración)

Tipos de control en PAM

- **auth** (Autenticación): Verifica la identidad del usuario.
- **account** (Gestión de cuentas): Comprueba si la cuenta del usuario tiene permiso para acceder en ese momento.
- **password** (Gestión de contraseñas): Se encarga de actualizar y cambiar las credenciales.
- **session** (Gestión de sesiones): Realiza tareas previas al inicio de sesión y posteriores al cierre (como montar directorios temporales o registrar accesos en los logs del sistema).

PAM (configuración)

Indicadores de control en PAM

- required (Obligatorio): El módulo debe tener éxito. Si falla continua resto de modulo, pero al final fallo.
- requisite (Requisito estricto):El módulo debe tener éxito obligatoriamente.
- sufficient (Suficiente): Si tiene éxito aprueba la autenticación de inmediato. Si no fallo un required anterior.
- optional (Opcional): El éxito o el fallo de este módulo no determina por sí solo el resultado final de la autenticación. Tareas secundarias.
- include (Incluir): Directiva que sirve para insertar el contenido de otro archivo de configuración de PAM.

PAM (Ejemplo módulos)

Nombre Modulo	Descripción
pam_unix.so	Tipo auth, account, password, session mecanismo clasico de Unix /etc/passwd y /etc/shadow. try_first_pass: si no hay token , pregunta. use_first_pass: si no hay, falla sin preguntar.
pam_google_authenticator.so	implementa Two-Factor Authentication (2FA) usando Google Authenticator.
pam_nologin.so	Solo pueden acceder usuarios si /etc/nologin no existe, en caso contrario solo el root podra entrar.
pam_securetty.so	Limita las consolas en las que se puede autentificar root /etc/securetty
pam_access	tipo account controla si un usuario o grupo de usuarios puede acceder /etc/security/access.conf
pam_time	Reduccion de servicios en franjas horarias

PAM (Programación)

Compilación

gcc -D_GNU_SOURCE -fPIC -DPIC -Wall -Wextra -shared -rdynamic -o <modulo.so>_<fuente.c> -lpam

- -fPIC Necesario librería compartidas, modulo sera cargado en otros módulos
- -shared Libreria compartida
- -rdynamic Añade todos los símbolos (funciones, variables)
- -lpam Enlaza la librería del sistema de PAM (libpam).

Cabeceras

#include <security/pam_modules.h>
#include <security/pam_ext.h>

Instalar

sudo cp ./bin/<modulo>.so /lib/security/<moludo>.so
sudo chown root:root /lib/security/<modulo>.so
sudo chmod 755 /lib/security/<modulo>.so

PAM (Prog. FuncBase)

PAM_EXTERN int <f_Cpntrol>
(pam_handle_t *pamh, int flags, int argc, const char **argv)

- *pam_sm_authenticate*: (auth) Verifica la identidad de usuario
- pam_sm_acct_mgmt: (account) validación de la cuenta
- pam_sm_open_session (session) Se ejecuta al iniciar sesión
- pam_sm_close_session (session Se ejecuta al cierre de sesión
- pam_sm_chauthtok (password) Actualiza o cambio de password
- pam_sm_setcred (auth) se ejecuta como parte de la fase de autenticación

Si no se usa **return PAM_SUCCESS;** (Exito)

PAM (Prog. Retorno)

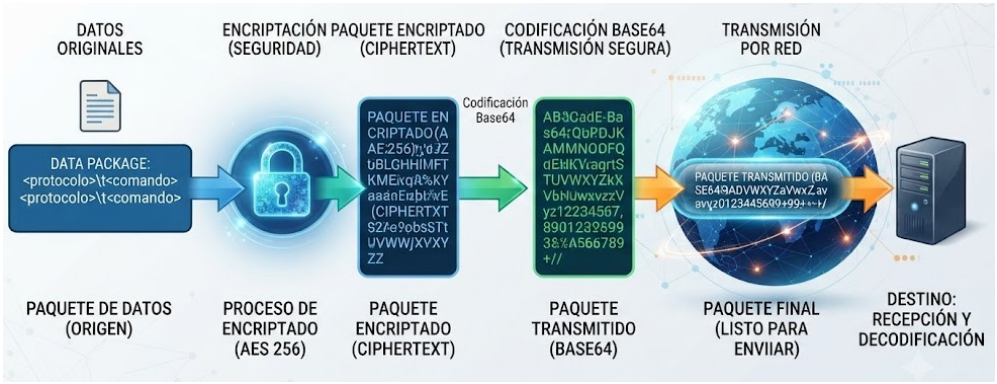
Códigos de retorno en PAM

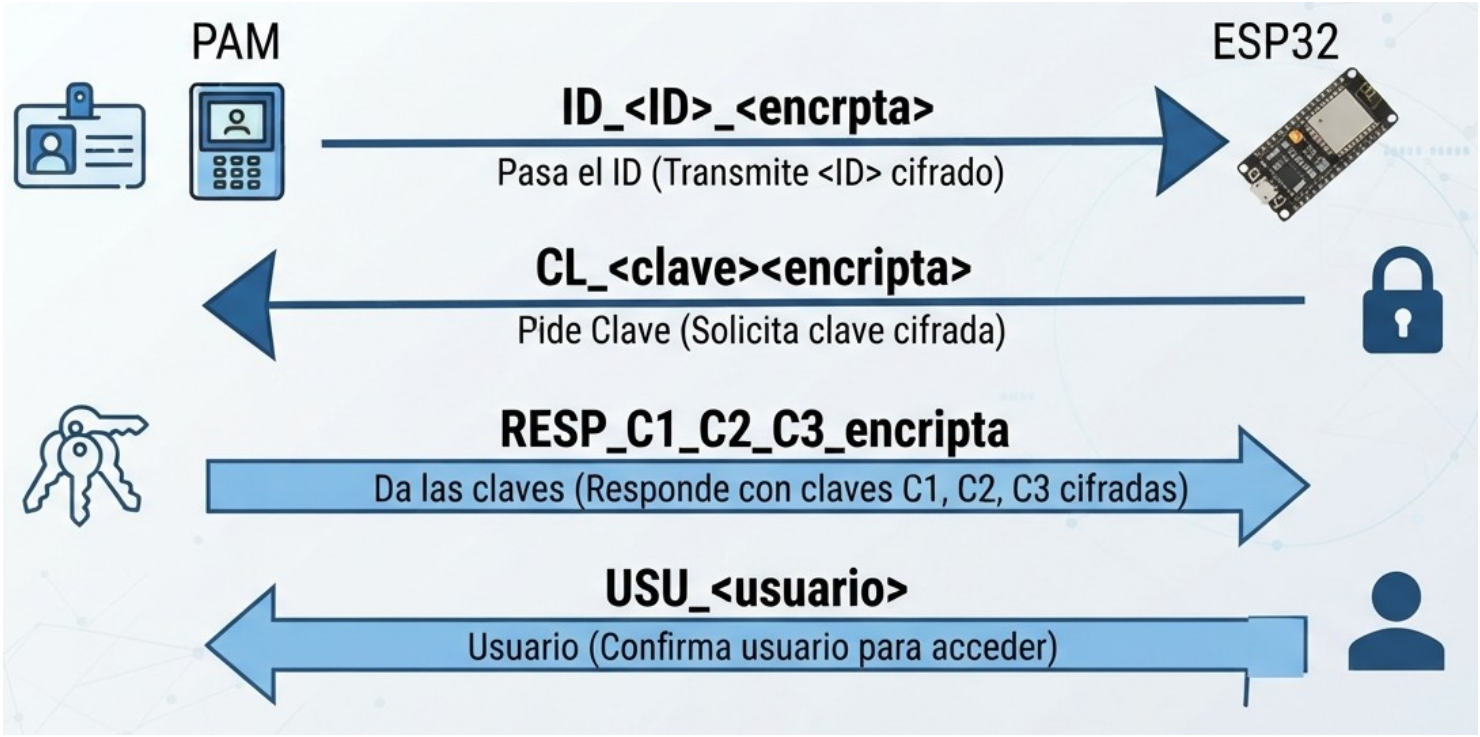
- PAM_SUCCESS (Éxito) El flujo avanza al siguiente módulo con normalidad.
- PAM_IGNORE (Ignorar) Pasa al siguiente como si no estuviera, sin afectar el resultado final.
- PAM_SERVICE_ERR (Error del servicio) aborta el proceso.
- PAM_AUTH_ERR (Error de autenticación) credenciales son inválidas.
- PAM_USER_UNKNOWN (Usuario desconocido) usuario no existe.
- PAM_ACCT_EXPIRED (Cuenta expirada) Cuenta caducada.
- PAM_PERM_DENIED (Permiso denegado) El usuario no tiene privilegios necesarios para realizar la acción solicitada.
- PAM_MAXTRIES (Intentos máximos superados)

PAM (Programación)

- pam_syslog(pamh, LOG_INFO, "Este es el servicio %s", servicio);
  journalctl | grep <modulo>
- pam_info(pamh, "Mensaje %d",valor);
- pam_get_item(pamh, TipoDato, (const void **)&dato);
  PAM_SERVICE
  PAM_TTY
  PAM_USER
  PAM_AUTHTOK
  PAM_RHOST
  PAM_RUSER
- pam_set_item(pamh, TipoDato, dato);

Protocolo


DATOS ORIGINALES
ENCRIPTACIÓN (SEGURIDAD)
PAQUETE ENCRIPTADO (CIPHERTEXT)
CODIFICACIÓN BASE64 (TRANSMISIÓN SEGURA)
TRANSMISIÓN POR RED
DATA PACKAGE:
<protocolo>\t<comando>
<protocolo>\t<comando>
Codificación Base64
PAQUETE DE DATOS (ORIGEN)
PROCESO DE ENCRIPTADO (AES 256)
PAQUETE ENCRIPTADO (CIPHERTEXT)
PAQUETE TRANSMITIDO (BASE64)
PAQUETE FINAL (LISTO PARA ENVIIAR)
DESTINO: RECEPCIÓN Y DECODIFICACIÓN


PAM
ESP32
ID_<ID>_<encrpta>
Pasa el ID (Transmite <ID> cifrado)
CL_<clave><encripta>
Pide Clave (Solicita clave cifrada)
RESP_C1_C2_C3_encripta
Da las claves (Responde con claves C1, C2, C3 cifradas)
USU_<usuario>
Usuario (Confirma usuario para acceder)



GRACIAS

https://github.com/pinguytaz/Ponencias